Today’s agenda

——----------

1- Revision
2- Docker Volume
3- Docker networking
4- Docker Push on DockerHub & AWS ECR

Step 1: Create the project folder

mkdir shopease
cd shopease

Dockerfile

```
# 1. Start from a small official Python image
FROM python:3.12-slim

# 2. Set the working directory
WORKDIR /app

# 3. Copy the dependency list first (improves build caching)
COPY requirements.txt .

# 4. Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy the application code
COPY app.py .

# 6. Set the application environment variable
ENV APP_ENV=dev

# 7. Document the application's port
EXPOSE 5000

# 8. Start the application
CMD ["python", "app.py"]
```

app.py (plz get the code/ indentation correct using chatgpt if required)

```python
from flask import Flask, jsonify
import os
import socket

app = Flask(__name__)

PRODUCTS = [
    {
        "id": 1,
        "name": "Wireless Mouse",
        "price": 799
    },
    {
        "id": 2,
        "name": "USB-C Cable",
        "price": 299
    },
    {
        "id": 3,
        "name": "Laptop Stand",
        "price": 1499
    }
]


@app.route("/")
def home():
    return jsonify(
        message="Welcome to ShopEase!",
        served_by=socket.gethostname(),
        environment=os.getenv("APP_ENV", "unknown")
    )
```

```python
@app.route("/products")
def products():
    print("INFO: /products was requested", flush=True)
    return jsonify(PRODUCTS)


@app.route("/health")
def health():
    return jsonify(status="ok")


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 3: Create requirements.txt

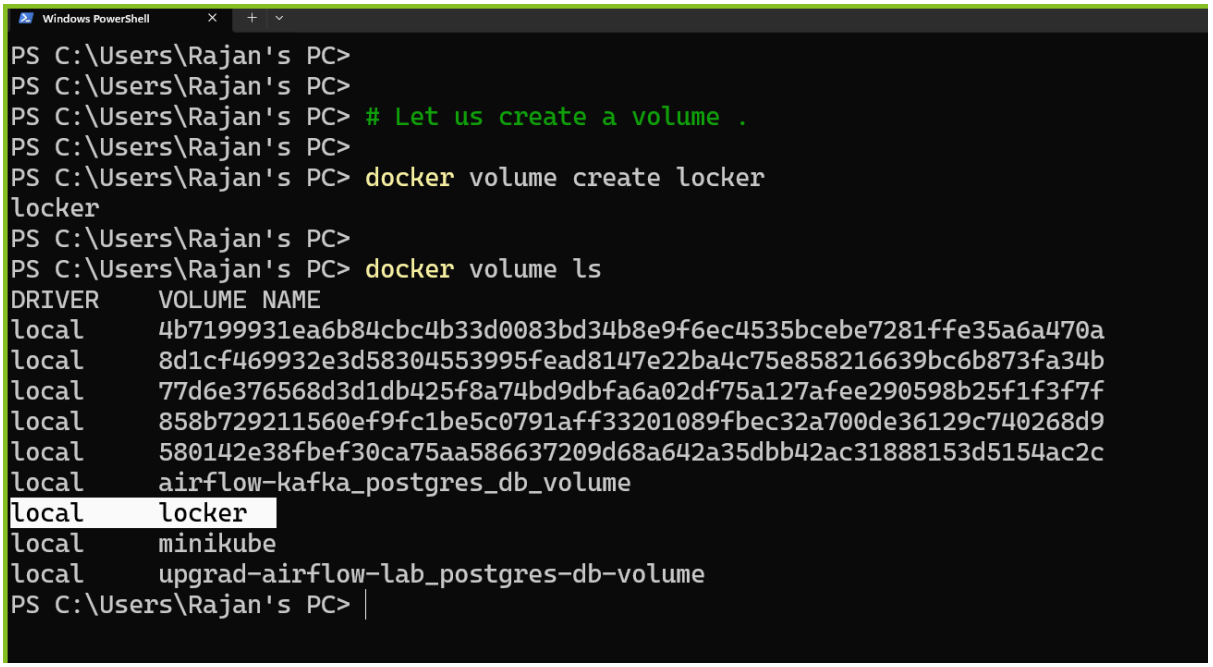
flask==3.0.3

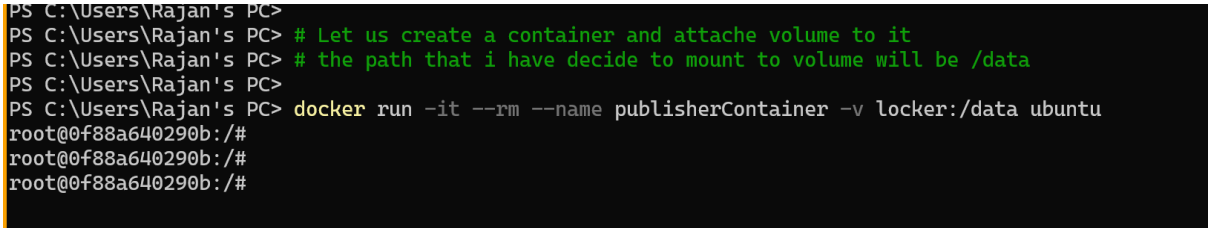
Step 6: Build the image

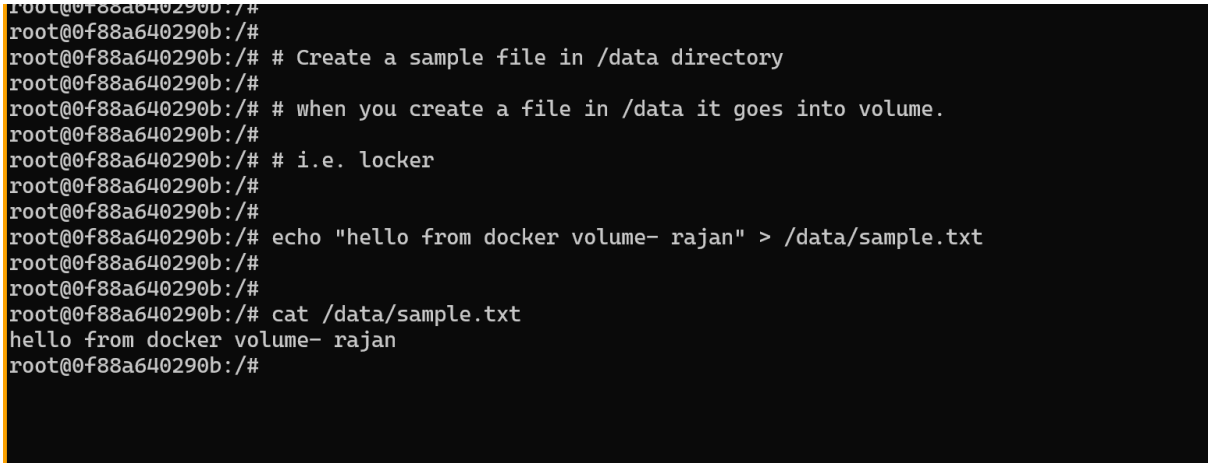
docker build -t shopease:1.0 .

Step 7 : Launch the Container & Expose Port 5000

Step 8 : Access the website & test Routes.
/health
/products


Windows PowerShell
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> # Let us create a volume .
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker volume create locker
locker
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker volume ls
DRIVER VOLUME NAME
local 4b7199931ea6b84cbc4b33d0083bd34b8e9f6ec4535bcebe7281ffe35a6a470a
local 8d1cf469932e3d58304553995fead8147e22ba4c75e858216639bc6b873fa34b
local 77d6e376568d3d1db425f8a74bd9dbfa6a02df75a127afee290598b25f1f3f7f
local 858b729211560ef9fc1be5c0791aff33201089fbec32a700de36129c740268d9
local 580142e38fbef30ca75aa586637209d68a642a35dbb42ac31888153d5154ac2c
local airflow-kafka_postgres_db_volume
local locker
local minikube
local upgrad-airflow-lab_postgres-db-volume
PS C:\Users\Rajan's PC>


PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> # Let us create a container and attache volume to it
PS C:\Users\Rajan's PC> # the path that i have decide to mount to volume will be /data
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker run -it --rm --name publisherContainer -v locker:/data ubuntu
root@0f88a640290b:/#
root@0f88a640290b:/#
root@0f88a640290b:/#


root@0f88a640290b:/#
root@0f88a640290b:/#
root@0f88a640290b:/# # Create a sample file in /data directory
root@0f88a640290b:/#
root@0f88a640290b:/# # when you create a file in /data it goes into volume.
root@0f88a640290b:/#
root@0f88a640290b:/# # i.e. locker
root@0f88a640290b:/#
root@0f88a640290b:/#
root@0f88a640290b:/# echo "hello from docker volume- rajan" > /data/sample.txt
root@0f88a640290b:/#
root@0f88a640290b:/#
root@0f88a640290b:/# cat /data/sample.txt
hello from docker volume- rajan
root@0f88a640290b:/#

```
root@0f88a640290b:/#
root@0f88a640290b:/# echo "hello from docker volume- rajan" > /data/sample.txt
root@0f88a640290b:/#
root@0f88a640290b:/#
root@0f88a640290b:/# cat /data/sample.txt
hello from docker volume- rajan
root@0f88a640290b:/#
root@0f88a640290b:/#
root@0f88a640290b:/# ls /data
sample.txt
root@0f88a640290b:/# exit
exit
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC>
```

Networking in Docker

Driver Type :

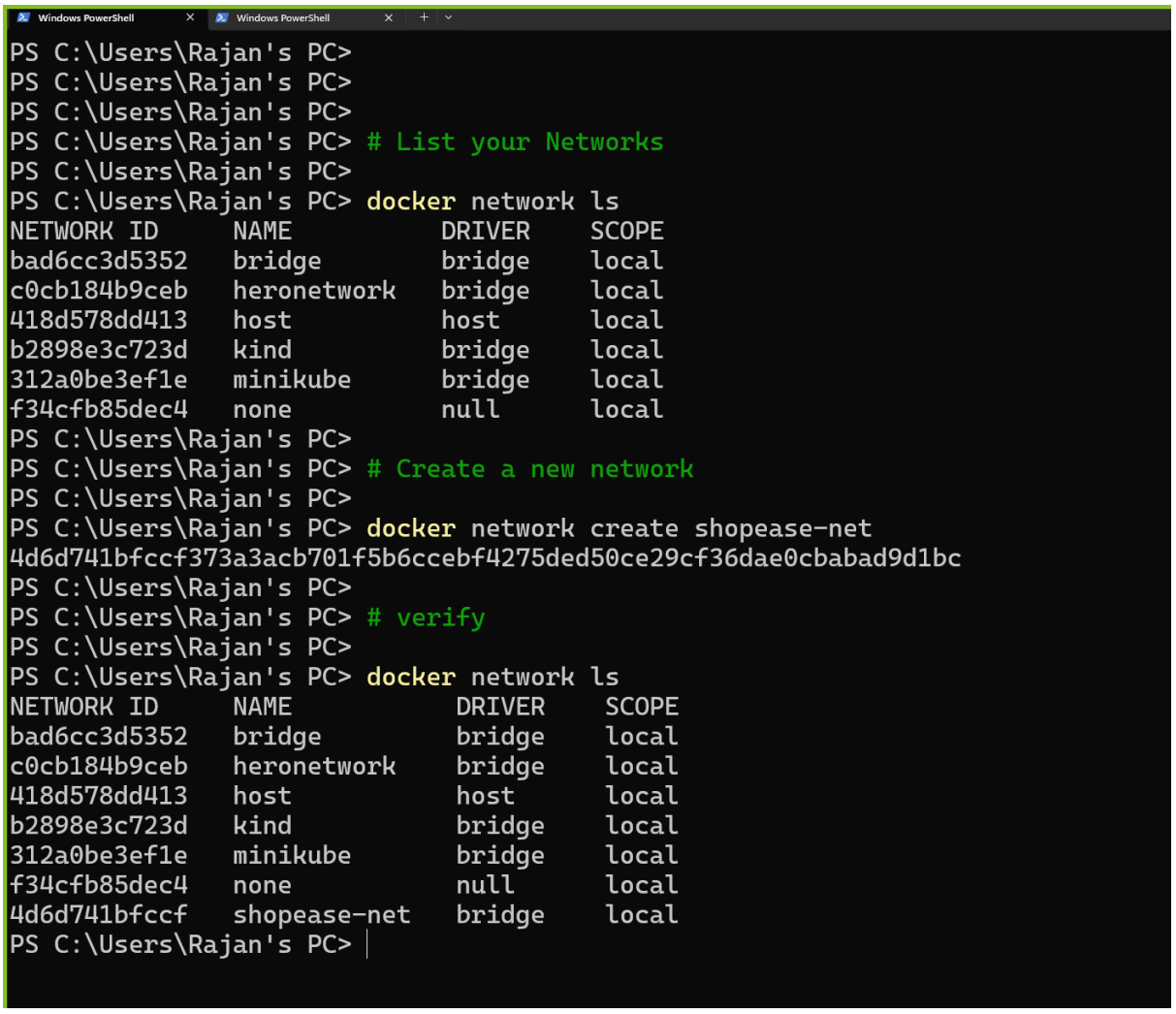
Bridge: it create a virtual Private Network (recall private Subnet in EC2 attached
To a NAT Gateway)

Host : host simply means your container shares the network of your host system directly.

None : simply no network

Overlay : when you wish to connect /establish connection between containers belonging to 2 different docker environments.

```
PS C:\Users\Rajan's PC> docker network ls
NETWORK ID     NAME          DRIVER    SCOPE
bad6cc3d5352   bridge        bridge    local
c0cb184b9ceb   heronetwork   bridge    local
418d578dd413   host          host      local
b2898e3c723d   kind          bridge    local
312a0be3ef1e   minikube      bridge    local
f34cfb85dec4   none          null      local
PS C:\Users\Rajan's PC>
```


Windows PowerShell
Windows PowerShell
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> # List your Networks
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker network ls
NETWORK ID NAME DRIVER SCOPE
bad6cc3d5352 bridge bridge local
c0cb184b9ceb heronetwork bridge local
418d578dd413 host host local
b2898e3c723d kind bridge local
312a0be3ef1e minikube bridge local
f34cfb85dec4 none null local
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> # Create a new network
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker network create shopease-net
4d6d741bfccf373a3acb701f5b6ccebf4275ded50ce29cf36dae0cbabad9d1bc
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> # verify
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker network ls
NETWORK ID NAME DRIVER SCOPE
bad6cc3d5352 bridge bridge local
c0cb184b9ceb heronetwork bridge local
418d578dd413 host host local
b2898e3c723d kind bridge local
312a0be3ef1e minikube bridge local
f34cfb85dec4 none null local
4d6d741bfccf shopease-net bridge local
PS C:\Users\Rajan's PC>

\

DOCKER NETWORKING

app2.py

```
from flask import Flask, jsonify
import os
import socket
import redis

app = Flask(__name__)

PRODUCTS = [
    {"id": 1, "name": "Wireless Mouse", "price": 799},
    {"id": 2, "name": "USB-C Cable", "price": 299},
    {"id": 3, "name": "Laptop Stand", "price": 1499},
]

REDIS_HOST = os.getenv("REDIS_HOST", "localhost")

cache = redis.Redis(
    host=REDIS_HOST,
    port=6379,
    decode_responses=True,
    socket_connect_timeout=2,
)


@app.route("/")
def home():
    return jsonify(
        message="Welcome to ShopEase v2!",
        served_by=socket.gethostname(),
        environment=os.getenv("APP_ENV", "unknown"),
    )


@app.route("/products")
def products():
    print("INFO: /products was requested", flush=True)
    return jsonify(PRODUCTS)


@app.route("/visits")
def visits():
    try:
        count = cache.incr("visits")
        return jsonify(
            visits=count,
            cache_host=REDIS_HOST,
        )
    except redis.exceptions.ConnectionError:
        print(
            f"ERROR: cannot reach Redis at {REDIS_HOST}:6379",
```

```
            flush=True,
        )
        return jsonify(error="cache unavailable"), 500


@app.route("/health")
def health():
    return jsonify(status="ok")


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
flask==3.0.3
redis==5.0.8
```

Dockerfile

```
# 1. Start from a small official Python image
FROM python:3.12-slim

# 2. Set the working directory
WORKDIR /app

# 3. Copy the dependency list first (improves build caching)
COPY requirements.txt .

# 4. Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy the application code
COPY app2.py .

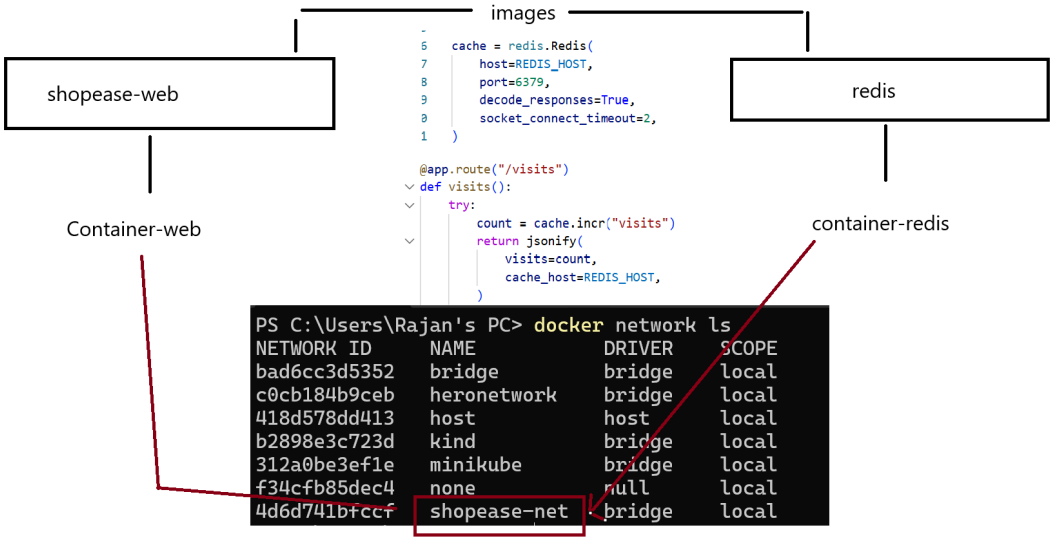
# 6. Set the application environment variable
ENV APP_ENV=dev

# 7. Document the application's port
EXPOSE 5000

# 8. Start the application
CMD ["python", "app2.py"]
```

Build a New Image with the Name shopease-web
docker build -t shopease-web .

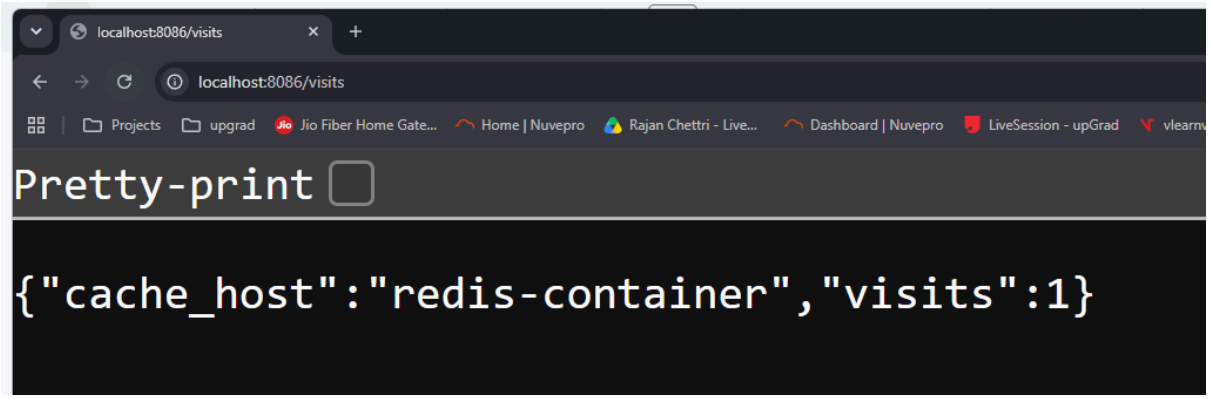
```
PS C:\Users\Rajan's PC\Downloads\dockerLabs> docker images
REPOSITORY                                                  TAG
shopease-web                                                latest
```

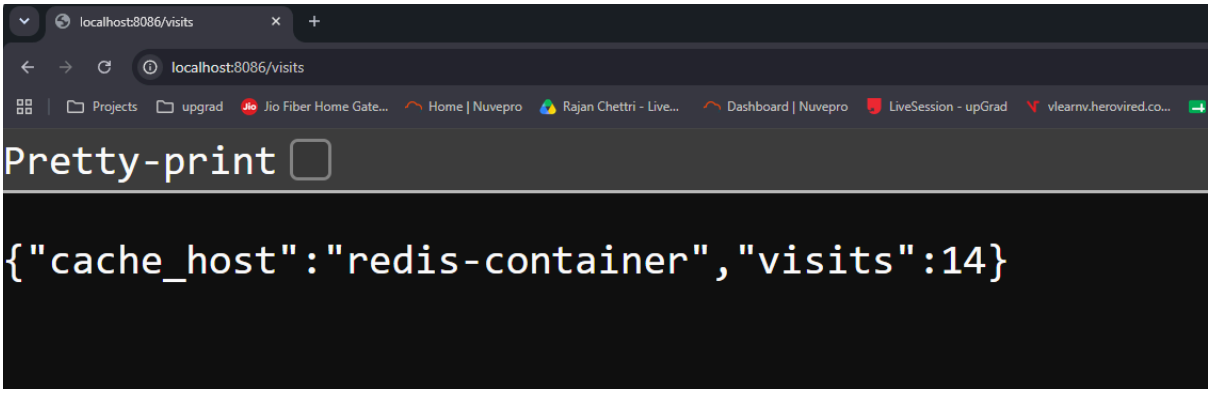



```
PS C:\Users\Rajan's PC> # Deploy the Redis Container using Redis Image on shopease-net Network.
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker run -d --name redis-container --network shopease-net redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
16333ee0c00f: Pulling fs layer
83234986c742: Pulling fs layer
7eb50503c2b0: Pulling fs layer
1522f9c6f453: Waiting
9928009b15c8: Waiting
4f4fb700ef54: Waiting
```

docker run -d --name shopease-container --network shopease-net -p 8086:5000 -e REDIS_HOST=redis-container shopease-web

```
PS C:\Users\Rajan's PC> docker run -d --name shopease-container --network shopease-net -p 8086:5000 -e REDIS_HOST=redis-container shopease-web
ef762a20cb9245e2321d68c985f34cf7e81be5b1557e4fcba2d34e71ce06a53a
PS C:\Users\Rajan's PC>
```





Pushing Docker Image Into Docker Hub & AWS ECR

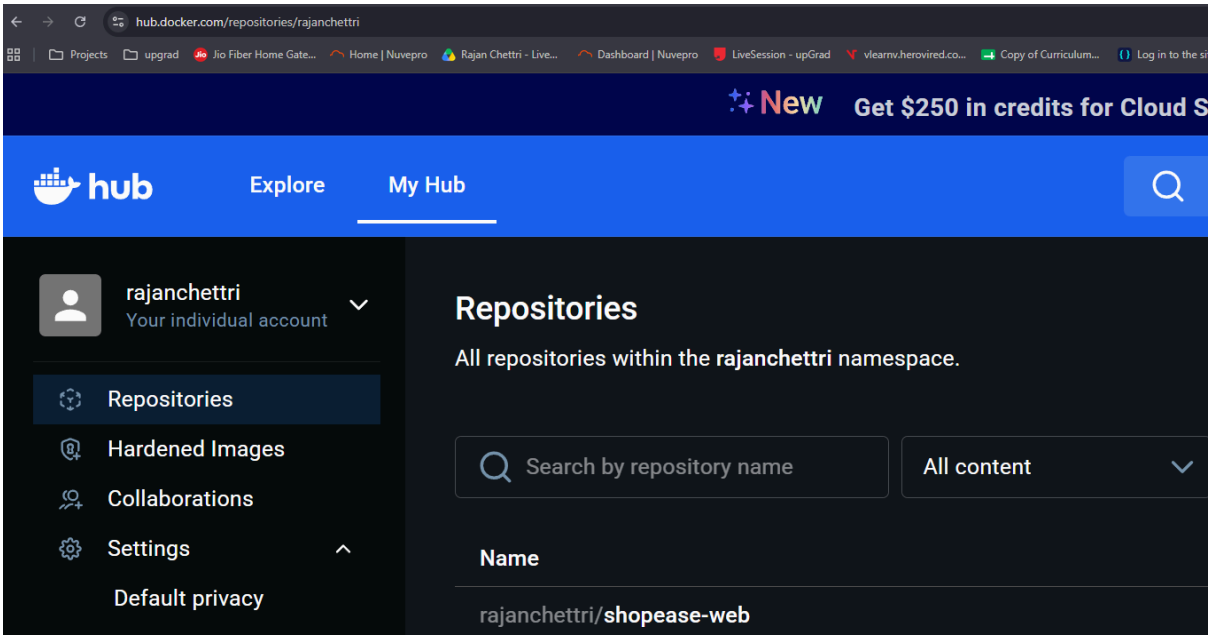
```
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker images
REPOSITORY                                                TAG
shopease-web                                              latest
858093957740.dkr.ecr.us-east-1.amazonaws.com/herorepo     latest
mynewnginximage                                           latest
rajanchettri/python                                       latest
python                                                    3.12.15-slim-trixie
nginx                                                     latest
ubuntu                                                    latest
redis                                                     7-alpine
PS C:\Users\Rajan's PC> docker tag shopease-web rajanchettri/shopease-web
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker images
REPOSITORY                                                TAG
rajanchettri/shopease-web                                 latest
shopease-web                                              latest
858093957740.dkr.ecr.us-east-1.amazonaws.com/herorepo     latest
mynewnginximage                                           latest
python                                                    3.12.15-slim-trixie
rajanchettri/python                                       latest
nginx                                                     latest
ubuntu                                                    latest
redis                                                     7-alpine
PS C:\Users\Rajan's PC>
```

```
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> # login into your docker account
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker login
Authenticating with existing credentials... [Username: rajanchettri]

i Info → To login with a different account, run 'docker logout' follo

Login Succeeded
PS C:\Users\Rajan's PC>
```

```
>> # ^C
PS C:\Users\Rajan's PC> # push image
PS C:\Users\Rajan's PC>
PS C:\Users\Rajan's PC> docker push rajanchettri/shopease-web
Using default tag: latest
The push refers to repository [docker.io/rajanchettri/shopease-web]
ffc45f1e4b92: Preparing
9a62440e579c: Preparing
```


hub.docker.com/repositories/rajanchettri
New
Get $250 in credits for Cloud S
hub
Explore
My Hub
rajanchettri
Your individual account
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Repositories
All repositories within the rajanchettri namespace.
Search by repository name
All content
Name
rajanchettri/shopease-web